

Voyage Project - Learning Path Generator

Tier 2 | Team Name: V62-T2-25 | September 2026

Table of Contents

- [A Note from Your Scrum Master](#)
- [Sprint Time Table](#)
- [Scrum Team & Goals](#)
- [Overview](#)
- [General Instructions](#)
- [Requirements & Specifications](#)
- [Stretch Goals](#)
- [Acceptance Criteria](#)

A Note from Your Scrum Master

Hi Team,

Thank you for the strong turnout at our kickoff on the 2nd of September. That kind of energy tells me we are starting this Voyage the right way: together, with everyone's voice in the room.

This document brings together two things you need over the next six weeks: the full project specification from Chingu, and the sprint timetable we agreed on for delivering it. Please read it once, start to finish, before Sprint 1 begins on the 7th, then keep it close as your reference throughout the Voyage.

A few reminders as we move from planning into execution:

- The Basic Requirements section is our Minimum Viable Product. That is our non-negotiable target for this Voyage. Everything in Stretch Goals is a bonus once the MVP is solid.
- The Acceptance Criteria at the end of this document is how we will know we are done. Refer back to it often, especially before marking a story complete.
- No one is expected to know everything on day one. Andresc is here as our Technical Guide, and we will use standups and reviews to surface blockers early rather than letting them sit.
- If anything in this document is unclear, say so. A question raised now saves rework later, and there are no bad questions on this team.

You were each brought onto this team because of the skills and goals you brought to the table. Trust the process, trust each other, and let's build something we are all proud to ship.

Onward,

Adewale & Apai/Francisco, Agile Champions

Sprint Time Table

Phase	Dates	Focus
Kickoff	Sep 2 (Tue)	Team intro, high attendance, initial alignment
Sprint 0	Sep 2 – 4	Setup, backlog grooming, tech stack and tooling decisions
Planning Day	Sep 4 (Fri)	Finalise backlog and sprint plan, ready for Sprint 1
Sprint 1	Sep 7 – 20 (2 wks)	Core AI generation build begins (user input and AI-powered path generation)
Sprint 2	Sep 21 – Oct 4 (2 wks)	Display, progress tracking, regenerate and start over

Sprint 3	Oct 5 – 11 (1 wk)	Polish, bug fixes, stretch goals if ahead of schedule
Sprint 4 (Harden & Ship)	Oct 12 – 18 (1 wk)	QA, deploy, documentation, demo

Notes

- Sprint 1 goal: user can submit their goal and context and receive an AI-generated path, with loading and error states handled.
- Sprint 4 is a dedicated harden-and-ship week, not shared with feature work, to reduce deployment risk.
- Stretch goals are only pulled into Sprint 3 if Basic Requirements (MVP) are fully met ahead of schedule.

Scrum Team & Goals

Web Developers	Agile Champions	Technical Guide
• Damaris Saavedra – Software Developer. Goal: continue developing experience focused on development, and hopefully land a full-time developer role soon. • Luna Dong – UX Designer. Goal: find a junior designer position remotely and gain more experience in design. • Matthew Neie – Software Developer. Goal: build skills and knowledge to find work as a junior developer. • Eleazer – Software Engineering. Goal: collaborate with the team and build relevant skills. • Nana – Backend Developer. Goal: collaborate with other developers, contribute to meaningful projects, and keep growing as a backend engineer. • Cruz – Software Developer. Goal: gain real-world experience and learn from the team.	• Adewale – Primary Scrum Master. Goal: guide the team in adopting the Scrum framework to design, build, test, and deploy the Learning Path Generator. • Apai / Francisco – Shadow Scrum Master. Goal: gain hands-on software experience while helping deliver a great Learning Path Generator.	Andresc – Technical Guide. Here to help with any technical issues you find along the way; reach out on this thread or by DM any time.

Overview

Hi, Chingus!

You've probably had that moment of staring at a new career goal and wondering: where do I even start? Whether it's breaking into web development, leveling up to a senior engineer, or pivoting into data science, the sheer number of tutorials, courses, and roadmaps out there can be overwhelming.

In this Voyage, your team will design and build a Learning Path Generator: a web application that uses AI to turn a person's career goals into a clear, personalized, step-by-step learning journey. Instead of guessing which skills to learn next, users will get a tailored path that adapts to where they are and where they want to go.

This is a great project because it combines thoughtful prompt engineering, full-stack development, and UI/UX design, all skills you'll use throughout your career as a web developer. You'll work with AI APIs to generate meaningful, structured

content, design an interface that makes complex information easy to navigate, and collaborate as a team to bring it all together.

Your goal is to create an app that's genuinely useful, easy to understand, and something you'll be proud to show off in your portfolio!

General Instructions

This project is designed to be worked on by a team rather than an individual Chingu. This means you and your team will need to thoroughly read and understand the requirements and specifications below, and define and manage your project following the Agile Methodology described in the Voyage Handbook.

As you create this project, make sure it meets all of the basic requirements. Once you've reached your Minimum Viable Product (MVP) state, start implementing the optional stretch goals, or get creative and extend it in ways we haven't even thought of yet. In other words, use the power of teamwork to make your version distinctive and unique.

A few things to keep in mind:

- UI/UX creativity is yours to explore: design an interface that stands out while still being easy to use. There is no single "right" way for the app to look.
- No paid subscriptions or software are required for this project. All tools and resources you need are free.
- You may use AI for research and learning. However, you may NOT use AI to write the code for your app. The code must be written by your team.

There are many web hosting services with free tiers you can use to deploy your app.

Requirements and Specifications

What You Need to Do

The sections below describe what your team needs to build. The Basic Requirements are the minimum features needed to have a working app (your MVP). The Stretch Goals are extra features for teams that finish early or want an extra challenge; they are completely optional.

Tier Note: Basic Requirements are designed for Tier 1 Voyagers (Beginner Frontend). Stretch Goals are designed with Tier 2 and Tier 3 Voyagers in mind, but any team is welcome to try them. A few individual items below are called out as required only for specific tiers; look for the tier note next to that item.

Make sure you control the scope so you'll be able to complete the basic requirements by the end of the Voyage. Remember, you can always add optional items and enhancements once you've completed the basic app.

Structure

- This can be implemented as a frontend application for Tier 1 and Tier 2. Tier 3 teams must implement both a frontend and a backend: see Backend / Full-Stack below for the minimum backend expectations.
- You may use any languages, tools, or libraries your team agrees on to design and build this app.
- Your app must be deployed to a hosting service so it can be accessed through the Internet. Free hosting options include GitHub Pages, Netlify, Vercel, or Render.
- Your GitHub repo must contain a well-written README.md file that describes your app and includes the link to the deployed version.
- Navigation options (as applicable):
 - Home: displays the landing page when clicked.
 - Results: displays the generated learning path, progress tracking, and any statistics your team decides to track.

- Screens:
 - Home Screen (a very basic example outline. Intentionally left vague because we want to see what you create):
 - Content that advertises the purpose of the app and its benefits.



Landing Page Wireframe

- *Optional:* user authentication is not required for the MVP; see Stretch Goals if your team wants to add it.
- Results Screen (design is intentionally left open for your team to define; it should surface the generated path and, where applicable, progress tracking).
- User Interface and Experience (UI/UX):
 - In general, you will find these UI design principles helpful.
 - If your team doesn't include a dedicated UI/UX Designer, you will find these tips helpful.

Styling

- Surprise us! Use your team's creativity to make this app distinctive.
- Your app should be visually clean and easy to read. Think about contrast, font size, and spacing so that all users can comfortably use the app.
- Every page (or screen) should include:
 - A header with the name of your app.
 - A footer with a link to your team's GitHub repository.
- Your app should look good on both desktop and mobile screen sizes (responsive design).
- If your team includes a UI/UX Designer, they should create wireframes for each screen before the team starts coding. Refer to the Chingu Handbook's UI/UX tips for guidance.
- Recommended resource for clean CSS.

Functionality

- Application Overview:
 - Teams will have to decide on how best to store data as needed. Browser Local Storage is a good option. You could also use free tiers from Firebase or Supabase. You can surprise us by adding more, as your team decides.

Basic Requirements (MVP)

These are the features your app must include. Think of them as the rules of the Voyage: without them, the app isn't really a Learning Path Generator.

User Input

- The app must provide a form or input flow where the user can enter their career goal (e.g., "Frontend Developer," "Data Scientist," "UX Designer").
- The app must collect at least a few pieces of context to personalize the path, such as:
 - The user's current skill level (e.g., Beginner, Intermediate, Advanced).
 - Any relevant background or existing skills (free text or a short list).
 - A time commitment or timeframe (e.g., "5 hours/week," "3 months").
- The app must validate user input before submission: for example, the career goal field should not be left empty.
- Input fields should have clear labels and placeholder text so users know what kind of information to enter.

AI-Powered Path Generation

- The app must send the user's input to an AI API to generate a personalized learning path.
- The generated learning path must be broken down into clear, sequential steps or milestones, not just a wall of text.
- Each step in the path must include, at minimum:
 - A title (e.g., "Learn HTML & CSS Basics").
 - A short description of what the step covers and why it matters.
 - An estimated time to complete the step.
- While the AI request is processing, the app must display a loading state (e.g., spinner or "Generating your path...") so the user knows the app is working.
- The app must handle and gracefully display an error if the AI request fails (e.g., "Something went wrong generating your path. Please try again.").

Displaying the Learning Path

- The generated path must be displayed in a clear, organized format: for example, a vertical roadmap, a numbered list, or a series of cards.
- Each step must be visually distinct so users can easily tell one milestone from the next.
- The path should clearly show progression and order (e.g., numbered steps, connecting lines, or a progress bar).

Tracking Progress *(Required for Tier 2 & 3; stretch goal for Tier 1)*

- Users must be able to mark a step as complete (e.g., checking it off).
- Completed steps must be visually marked as done (e.g., checkmark, strikethrough, or color change) so progress is easy to see at a glance.
- The app must display an overall progress indicator (e.g., "3 of 8 steps completed" or a progress bar) that updates as steps are checked off.

Regenerating & Starting Over

- After a path is generated, the user must have the option to generate a new path with different input (new goal, skill level, etc.).
- The app must clear or replace the old path when a new one is generated, so the user isn't looking at stale data.

Input Validation & Error Handling

- The app should not allow users to submit the form with missing required fields (e.g., no career goal entered). Display a clear message explaining what's missing.
- Error messages must be clear and helpful, not just "Error." For example: "Please enter a career goal before generating your path."
- Error messages should disappear or be cleared once the user corrects their input.
- If the AI returns an unexpected or malformed response, the app must handle it gracefully rather than crashing or showing a blank screen.

Stretch Goals

Once you complete the basic application, you may enhance it with any of the following optional stretch goals. Make sure that any of these you choose matches the capabilities of your team.

These features are designed with Tier 2 and Tier 3 skill levels in mind, but any team is welcome to try them. Make sure to add acceptance criteria to your repo's README.md for any stretch goals you implement.

Account, Personalization & AI Q&A

- Authenticate users via Google or GitHub to enhance your app's security.
- The app is already AI powered: use it to provide users with follow-up Q&A on any step in their path (e.g., "Why do I need to learn this?" or "Suggest an alternative resource"). The Gemini Flash 1.5 free tier is sufficient for this.

Enhanced Path Generation

- Skill Assessment Quiz: before generating a path, offer the user a short quiz to more accurately gauge their current skill level instead of relying on self-reported input.
- Multiple Path Variations: generate 2–3 different path options (e.g., "Fast Track," "Balanced," "In-Depth") based on the same goal, and let the user pick the one that fits their pace.
- Resource Recommendations: for each step in the path, have the AI suggest specific free or paid resources (courses, articles, videos, documentation) rather than just a description.
- Prerequisite Detection: allow the AI to flag prerequisite knowledge the user may be missing based on their stated background, and insert foundational steps automatically.
- Regenerate a Single Step: let users regenerate or refine just one step in the path (e.g., "make this step easier" or "give me a different resource") without regenerating the entire path.

Animations & Visual Polish

- Add smooth transition animations when a step is marked complete or when a new path is generated.
- Add a loading animation (beyond a simple spinner) while the AI generates the path, to make the wait feel more engaging.
- Add a color theme option, allowing the user to switch between a Light Mode and a Dark Mode for the app interface. Save the user's preference using localStorage so it persists between sessions.
- Add a visual roadmap/timeline view (e.g., a winding path or vertical timeline with icons) as an alternative to the default list/card view.

Progress & Sharing

- Progress Dashboard: track and display the user's stats across multiple learning paths, including:
 - Total paths generated.
 - Total steps completed.
 - Current streak (consecutive days with a completed step).

- Overall completion percentage per path.
- Store the statistics in the browser using localStorage so they persist between sessions.
- Share Progress: give the user a "Share" button that generates a shareable summary or image of their learning path progress (e.g., "I've completed 6/10 steps toward becoming a Frontend Developer!").
- Export Path: allow users to export their learning path as a PDF or markdown checklist they can save or print.

Accessibility

- Make the app fully keyboard accessible: every action (submitting the form, marking a step complete, generating a new path, opening settings) should be achievable without a mouse.
- Add screen reader support by using proper ARIA labels and roles on form inputs, step cards, and progress indicators, so that users who use assistive technology can understand and interact with the app.

Responsive Design

- Ensure the full app is fully usable on mobile devices (small touchscreens), not just desktop browsers. The roadmap/step layout should adapt cleanly to narrow viewports.

Backend / Full-Stack *(Required for Tier 3)*

- Build a backend server (e.g., using Node.js/Express) to:
 - Store generated learning paths in a database instead of only in local browser storage.
 - Store and retrieve user progress server-side, so it persists across devices and browsers.
- Implement user authentication (e.g., via GitHub OAuth or Google Sign-In) so users can log in and access their saved learning paths from any device.
- Allow users to save multiple learning paths (e.g., one for "Frontend Developer" and another for "Data Analyst") and switch between them from a dashboard.

Acceptance Criteria

Your finished project should meet all of the following minimum standards:

- Your GitHub repo contains a well-written README.md that includes a link to your deployed app, a description of the project, and the names/GitHub profiles of all team members.
- The deployed app is accessible and fully usable in a modern web browser without any console errors.
- The form validates required input (e.g., career goal) before submission, and displays clear, specific error messages when input is missing or invalid.
- The generated learning path is displayed in a clear, organized format with each step visually distinct and progression/order clearly indicated.
- A loading state is shown while the AI request is processing, and a clear error message is shown if the AI request fails.
- *(Tier 2 & 3)* Users can mark steps complete, completed steps are visually marked, and an overall progress indicator updates accordingly.
- Users can regenerate a new path with different input, and the old path is cleared or replaced rather than left on screen alongside the new one.
- The UI is responsive and usable on both desktop and mobile screen sizes.
- If your team implemented any stretch goals, add their acceptance criteria directly to your repo's README.md.